Unified Message Passing for Capacitated Clustering and Routing: CAP, the Spanning Forest, and the Trellis on One Factor Graph

Abstract

We study capacitated clustering-and-routing for waste collection (partition the bins into capacity-bounded zones, each served by one vehicle's optimal tour). Two clustering philosophies compete: capacity-constrained affinity propagation (CAP), which attaches every node *directly* to a representative depot (a star, minimising the 1-median cost), and the capacitated spanning *forest*, which attaches every node to its *nearest connected neighbour* (a chain, minimising the MST cost). On total travel the forest wins; on zone compactness and per-vehicle makespan CAP wins. We show these are not two algorithms but *one* model—a capacitated rooted forest expressed as max-sum on a single factor graph—differing only in the maximum tree depth L: L=1 is exactly CAP, L=∞ is exactly the forest, and adding node drops recovers prize-collecting Steiner clustering. We give the factor graph and its messages (the capacity factor reusing CAP's Nemhauser–Ullmann/Pareto knapsack), prove the two reductions, add a consistency factor that couples the trellis router back into the clustering as an effective similarity $\hat{s}(i,k) = s(i,k) + \tilde{\tau}_{ik}$, and validate the depth spectrum empirically on an 84-bin Seongbuk network. We also report two honest negative/again results: (i) CAP's tour-blindness is an implementation artefact of its greedy decode, removed by an argmax decode; (ii) feeding tour information back into a well-converged CAP does not improve it—the star objective is already solved, and connectivity is better obtained by moving L, not by perturbing CAP.

Contents

1 Introduction **2**

2 Background **2**

3 A rooted-forest factor graph **2**

4 CAP and the forest are the same model **3**

5 One unified framework **3**

6 Coupling the router into the clustering **4**

7 Baselines and evaluation protocol **4**

8 Experiments **5**

8.1 Decode: CAP's tour-blindness is a decode artefact 6

8.2 Structure: CAP is compact and low-makespan; forest is short in total 6

8.3 The depth dial: one knob trades total for radius 6

8.4 An honest negative: coupling does not beat converged CAP 6

9 Discussion and scope **7**

1 Introduction

A fleet of vehicles must service N spatially distributed bins from depots, each vehicle carrying at most Q. The task splits into *clustering* (assign bins to capacity-bounded zones with a depot) and *routing* (an optimal tour per zone). This note unifies the two clustering methods used in our pipeline—CAP and the spanning forest—and the exact per-zone router (the trellis) into a single message-passing framework, and characterises exactly when and why each clustering is preferable.

Our contributions:

1. **One model** (§3–5): a capacitated rooted-forest factor graph whose single depth knob L interpolates CAP (L=1, star) and the forest (L=∞, MST), and whose drop/opening knobs recover facility location and prize-collecting Steiner clustering.
2. **Router coupling** (§6): a consistency factor that injects the trellis's soft output into the clustering as $\hat{s} = s + \tilde{\tau}$.
3. **Evidence** (§8): the depth spectrum is confirmed on Seongbuk; CAP is quantitatively more compact and lower-makespan while the forest is shorter in total; the decode fix and the coupling result are reported honestly.

2 Background

Data. $V^+ = \{0, \ldots, N\}$ (depot = 0), distances $d(i,j) \geq 0$, weights w_i, per-tree capacity Q; $s(i,j) = -d(i,j)$, and $s(k,k) = p_k$ the preference (opening reward).

CAP. CAP assigns each node to an exemplar c_i to maximise $\sum_i s(i, c_i)$ subject to $\sum_{i:c_i=k} w_i \leq Q$. Its max-sum messages (net form) are

$$\tilde{\rho}_{ik} = s(i,k) - \max_{k' \neq k}\big(\tilde{\alpha}_{ik'} + s(i,k')\big), \tag{1}$$

$$\tilde{\alpha}_{ik} = \begin{cases} \mathcal{P}_k^{-\{k\}}(Q - w_k), & i = k, \\ \tilde{\rho}_{kk} + \mathcal{P}_k^{-\{i,k\}}(Q - w_i - w_k) - \big[\tilde{\rho}_{kk} + \mathcal{P}_k^{-\{i,k\}}(Q - w_k)\big]^+, & i \neq k, \end{cases} \tag{2}$$

where $\mathcal{P}_k^{-A}(L) = \max\{\sum_{j \notin A} r_j z_j : \sum_j w_j z_j \leq L\}$, $r_j = [\tilde{\rho}_{jk}]^+$, is the Nemhauser–Ullmann knapsack (built once per exemplar as a Pareto frontier). CAP is thus the belief-propagation solver of capacitated facility location / k-medoids.

Trellis. A cluster's optimal closed tour is a hypercube trellis (Held–Karp as min-sum) with state $q_t = (m_t, a_t)$ (visitation memory + current node), forward/backward metrics

$$\psi_t(q_t) = \max_{q_{t-1}}[G_t(q_{t-1}, q_t) + \psi_{t-1}(q_{t-1})], \quad \beta_t(q_t) = \max_{q_{t+1}}[G_{t+1}(q_t, q_{t+1}) + \beta_{t+1}(q_{t+1})], \tag{3}$$

whose combination $\psi_t + \beta_t$ is the SOVA soft output used in §6.

3 A rooted-forest factor graph

A *rooted forest* is a parent map $\pi : V \to V \cup \{\perp\}$ with acyclic functional graph; $\pi_i = \perp$ marks a root (depot). We maximise

$$\max_{\pi} \sum_{i:\pi_i = \perp} p_i + \sum_{i:\pi_i \neq \perp} s(i, \pi_i) \quad \text{s.t.} \quad \sum_{i \in T_k} w_i \leq Q \;\; \forall \text{ roots } k, \tag{4}$$

i.e. minimise the total *parent-attachment* length $\sum_i d(i, \pi_i)$ plus opening penalties.

Variables and factors. Per node i: parent $\pi_i \in V \cup \{\perp\}$, depth $\ell_i \in \{0, \dots, L\}$, and subtree load u_i.

$$U_i(\pi_i) = \begin{cases} p_i & \pi_i = \perp \\ s(i, \pi_i) & \text{else,} \end{cases} \qquad A_i = \begin{cases} 0 & \pi_i = \perp, \ell_i = 0 \\ 0 & \pi_i = j,\ \ell_i = \ell_j + 1 \le L \\ -\infty & \text{else,} \end{cases} \tag{5}$$

$$B_i : \ u_i = w_i + \sum_{j:\pi_j = i} u_j, \qquad E_k : \ \pi_k = \perp \Rightarrow u_k \le Q. \tag{6}$$

The depth factor A_i turns the *global* "acyclic and root-connected" constraint into a *local* one (depths strictly decrease to a depth-0 root), and B_i, E_k accumulate and cap subtree weight. The joint objective is $\max \sum_i [U_i + A_i + B_i + E_i]$.

Messages. In cavity (loopy BP) form, the node-to-parent message carrying depth ℓ and load u is

$$M_{i \to j}(\ell, u) = s(i, j) + F_{i \backslash j}(\ell + 1, u), \qquad F_{i \backslash j}(\ell, u) = \max_{\substack{S \subseteq \mathcal{C}_i \backslash \{j\} \\ w_i + \sum_c u_c = u \le Q}} \sum_{c \in S} \max_{u_c} M_{c \to i}(\ell, u_c), \tag{7}$$

where the subtree aggregation F is a 0/1 knapsack over the children's load–utility profiles, solved by the *same* Pareto frontier as CAP's availability (2). The root option adds p_i at $\ell_i = 0$; the decoded forest is $\pi_i^\star = \arg\max_{\pi_i} b_i(\pi_i)$.

4 CAP and the forest are the same model

Theorem 1 (Depth-1 = CAP)**.** *With $L = 1$, (4) is CAP: every non-root node attaches directly to a root, giving $\max \sum_i s(i, c_i)$ s.t. $\sum_{i:c_i = k} w_i \le Q$.*

Proof. L=1 allows depths $\{0, 1\}$; by A_i a non-root i has ℓ_i=1 and its parent has depth 0, hence is a root. So every tree is a depth-≤ 1 star, π_i is CAP's exemplar c_i, U_i gives $\sum_i s(i, c_i)$, and B_i, E_k reduce to the per-exemplar capacity with the same Pareto knapsack. □ □

Theorem 2 (Depth-∞ = capacitated MST forest)**.** *With $L = N$ and fixed roots, dropping E_k makes (4) a minimum spanning forest ($\min \sum_i d(i, \pi_i)$); reinstating E_k gives the capacitated spanning forest.*

Proof. Without E_k, (4) is $\min \sum_i d(i, \pi_i)$ subject only to A_i (acyclicity to a root), which is the min-parent-cost characterisation of the MST (per root, a spanning tree); E_k restores the per-tree capacity. □ □

Proposition 1 (CAP–forest spectrum)**.** *For $1 \le L \le N$, (4) interpolates the star (L=1, cost $\sum_i d(i, \text{root})$) and the MST (L=∞, cost $\sum_i d(i, \text{parent})$); increasing L lowers total attachment length at the cost of larger depot radius.*

5 One unified framework

Adding an optional drop set $\mathcal{D}$ with penalties π_i gives the *capacitated, hop-bounded, prize-collecting rooted forest*

$$\min_{\pi, \mathcal{D}} \sum_{i \notin \mathcal{D}} d(i, \pi_i) + \sum_{k \text{ open}} f_k + \sum_{i \in \mathcal{D}} \pi_i \quad \text{s.t.} \quad \sum_{i \in T_k} w_i \le Q, \quad \ell_i \le L, \tag{8}$$

with four interpretable knobs:

knob	meaning	small →	large →
L	depth (hops to depot)	L=1: **CAP** (compact)	L=∞: **forest** (short total)
f_k	depot opening cost	many clusters	few clusters
π_i	drop penalty	**PCST** (skip low-priority)	serve all
Q	capacity	many small trees	few large trees

By Theorems 1–2, L moves the solution along a total-length↔compactness Pareto front, so the operator selects the clustering by the objective actually in play: minimise fuel (L large), enforce depot-centred disruption-robust zones (L small), balance makespan (moderate L, Q), or skip nodes under a time budget (finite π_i). CAP is the star (depth-1, no-drop) corner of the prize-collecting family; a literal *max-radius* (1-center) target enters as a fifth knob $d(i, \mathrm{root}) \le \bar{r}$.

6 Coupling the router into the clustering

The clustering above optimises attachment cost, not tour length. To make it tour-aware without re-deriving CAP, attach one trellis per cluster and couple it to the assignment x_{ik} (= 1 iff node i is served by cluster k) through a consistency factor

$$K_{ik}(x_{ik}, v_{ik}) = \begin{cases} 0 & x_{ik} = v_{ik} \\ -\infty & \text{else,} \end{cases} \tag{9}$$

where v_{ik} is the trellis's visitation of i by cluster k. Being an equality gate, K_{ik} passes the difference message unchanged, so the router influences the clustering through a single scalar per pair,

$$\tilde{\tau}_{ik} = \Big[\max_{t,m}(\psi_t^{(k)} + \beta_t^{(k)})\big|_{a_t=i}\Big] - \Big[\max_{t,m,a_t \neq i}(\psi_t^{(k)} + \beta_t^{(k)})\Big], \tag{10}$$

the SOVA soft output of (3): the marginal routing value of admitting i into k's tour. Every CAP update then runs on the *effective similarity*

$$\boxed{\hat{s}(i,k) = s(i,k) + \tilde{\tau}_{ik}} \tag{11}$$

i.e. (1)–(2) with $s \to \hat{s}$; the knapsack prizes $[\tilde{\rho}_{jk}]^+$ then trade capacity against routing detour jointly. Setting $\tilde{\tau} \equiv 0$ recovers plain CAP.

7 Baselines and evaluation protocol

Because the framework has two separable components—clustering (§3–5) and routing (the trellis)—a fair comparison must fix one and vary the other; mixing them makes the numbers uninterpretable. We therefore organise baselines by the *claim* being tested, along four axes.

axis (claim)	baselines	metrics
A. Routing (exact tour) *fix clustering*	nearest-neighbour (NN), NN+2-opt, genetic algorithm (GA), brute force (small zones) — **routed on the same clusters**	total length, optimality gap vs BF
B. Clustering *fix router = exact trellis*	k-means / k-medoids + capacity repair, sweep algorithm, capacitated k-means, Jain–Vazirani facility location, the forest ($L=\infty$), geographic/random partition (floor)	total, **makespan**, max radius/diameter, load balance, feasibility
C. End-to-end *whole pipeline*	OR-Tools CVRP, LKH-3, Clarke–Wright savings, sweep+2-opt	total length, runtime, feasibility
D. Dynamic *mid-route disruption, budget T_c*	static plan, NN, NN+2-opt, GA — each re-optimising per step	cumulative cost over time, fallback frequency vs T_c

Axis A is the routing comparison of the online-routing framework (NN/2-opt/GA/BF) and isolates the exact trellis; axis B tests the clustering only and places CAP, the forest, and facility location as *points of the same dial* (§5) alongside external clusterers; axis C bounds absolute tour quality against state-of-the-art CVRP solvers; axis D is the framework's primary setting—adaptive re-solving under disruption—where soft message passing avoids premature hard decisions.

Three pitfalls (learned the hard way).

1. *Do not give the baselines the exact trellis.* On axis A/D the baselines must route with their own (NN/2-opt/GA) heuristics; handing them the trellis erases precisely the routing advantage being claimed.
2. *Do not frame on total distance alone.* As §8 shows, CAP loses total to the forest yet wins makespan and compactness; a strong CVRP solver (axis C) will also beat total. Report the multi-metric set (total, makespan, radius, robustness).
3. *Frame CAP vs. forest as two points of one dial, not rivals.* By Theorems 1–2 the forest winning on total is not a weakness of the framework but a selectable operating point (L large); the compact regime (L=1) is another.

Recommended minimal suite. Axis A with NN/2-opt/GA/BF on fixed clusters (routing optimality); axis B with k-means+repair, sweep, facility location and the forest under the multi-metric set (clustering); axis C with OR-Tools/LKH-3 as an absolute-quality reference; and axis D under the disruption model as the headline. The claim should be led by axes D and B (adaptivity, makespan, compactness), not by axis C total distance, which state-of-the-art CVRP solvers are expected to win.

8 Experiments

Seongbuk-gu, N=84 bins, OSM road distances, depot node 0, Q=12 unless noted. "uniform" weights = 1; "random" weights $\in \{1, \ldots, 9\}$ (a capacity-binding regime). Intra-cluster tours are the exact trellis.

8.1 Decode: CAP's tour-blindness is a decode artefact

CAP's message passing uses the correct (Pareto-verified) knapsack, but its integer *decode* matters. The original greedy capacity-repair fragments badly under tight capacity; the standard argmax decode (with minimal repair of the rare overloaded cluster) is far better:

random, Q=12	clusters	star cost	total tour
CAP, greedy decode	41	30,371	97,334
CAP, argmax decode	42	19,719	76,196
forest (L=∞)	45	15,497	70,231

The argmax decode is used below.

8.2 Structure: CAP is compact and low-makespan; forest is short in total

With near-equal cluster counts:

	total	max radius	max diameter	makespan (max route)	singletons
uniform, Q=12 (CAP k=12, forest k=8)					
CAP+trellis	**62,837**	**1,768**	**2,033**	**5,596**	0
forest	64,920	3,627	4,225	11,800	0
random, Q=12 (CAP k=42, forest k=45)					
CAP+trellis	77,739	**1,162**	**1,359**	**2,718**	**12**
forest	**70,231**	1,755	1,820	3,883	18

Forest minimises *summed* travel; CAP minimises distance to a centre and hence yields smaller worst-case radius/diameter, a 30–110% shorter *makespan* (latest-vehicle route), and fewer wasted singleton vehicles. In parallel multi-vehicle collection makespan and per-zone reach are often the operative metrics, so total-distance dominance does not settle the comparison.

8.3 The depth dial: one knob trades total for radius

Fixing the depots and sweeping L (greedy realisation, §3) confirms Proposition 1: the tree attachment cost $\sum_i d(i, \pi_i)$ falls monotonically from the star (L=1, CAP) to the MST (L=∞, forest), while the mean depot radius rises.

uniform, Q=24 (4 depots)	L=1	L=2	L=3	L=5	L=∞
tree cost $\sum d(i, \pi_i)$	87,174	66,433	52,496	38,733	31,021
mean depot radius	2,073	2,175	2,254	2,402	2,411

(At Q=48, 2 depots: tree cost 149,520 $\to$ 39,836 as L:1 $\to$ ∞.) A single integer dial thus navigates the CAP–forest spectrum.

8.4 An honest negative: coupling does not beat converged CAP

The coupling (11) was implemented as a coordinate ascent (run CAP on $\hat{s}$; recompute $\tilde{\tau}$; damp; repeat). Under a tight CAP iteration budget it improves the (under-converged) baseline, but with a proper budget the feedback does *not* beat plain CAP: the best round is the zeroth (no feedback), and subsequent rounds are worse.

random, Q=12, iter 200	round 0 (=CAP)	round 1	round 2	best
total tour	77,739	90,407	82,686	**77,739**

Interpretation: CAP already solves its star objective well; perturbing it with a routing marginal only pushes it off that optimum. If tour-aligned clusters are wanted, the principled route is to *change the objective* (increase L toward the forest), not to nudge CAP.

9 Discussion and scope

What is unified. CAP (assignment), the spanning forest (connectivity), and the trellis (routing) are all max-sum on one factor graph; CAP versus forest is the depth bound L, not a change of algorithm; facility location and prize-collecting Steiner clustering are the L=1 and drop-enabled corners of (8).

Honest caveats. (i) The depth sweep validates the *structural* objective of (4) (tree attachment cost vs radius); the trellis tour is re-optimised per partition, so L controls clustering geometry and the tour follows. (ii) The joint graph is loopy, so BP on the capacitated forest is heuristic (as for Steiner-tree BP); exactness is retained only in the two sub-modules—the Pareto knapsack and, at L=1, CAP's bipartite assignment. A single generic BP solver therefore does not match the dedicated CAP/forest solvers' quality at the extremes; the framework's value is the unification, the provable reductions, and access to intermediate operating points. (iii) The coupling of §6 is derived and implemented but does not improve a converged CAP; we report it as a negative result.

Takeaway. For parallel multi-vehicle collection the right clustering is objective-dependent: total travel favours large L (forest); compactness, bounded reach, low makespan, and disruption robustness favour L=1 (CAP). The unified rooted-forest factor graph exposes this as one dial, on the same message-passing footing as the router.